Optimizing Classification Capability of Classical Machine Learning Models on Traffic Signs

Deomary Angelo Franco (2023-22049)

Date of Submission: May 28, 2025

ABSTRACT

With the great trend in self-driving cars, the problem of object detection has been a major research focus. One of these is traffic sign recognition. While different deep learning models that use advanced techniques such as CNN are popular, it is also of interest if it is possible for classical machine learning methods to rival their performances. One paper shows that using a CNN can achieve a remarkable 98.98%. This study attempts to tackle this problem by generating hand-crafted features and seeing how far they can match.

This study focuses on optimizing a traffic sign classification model using models such as Stochastic Gradient Descent, Multi-Layer Perceptron, Support Vector Machines, and XGBoost. It compares their performance with other state-of-the-art models. Here, the German Traffic Sign Recognition Benchmark (GTSRB) dataset is utilized for training and testing. Different hand-crafted features are generated from the images such as Histogram of Gradients and Local Binary Pattern.

The result exhibits the usage of the flattened original dataset, combined with the Local Binary Pattern and the Histogram of Gradients as features, and then trained on a Support Vector Classifier with Radial Basis Function kernel achieved an impressive 99.09% accuracy and 99.04% F1 score. This manifests that while deep learning methods have their strengths, classical machine learning methods coupled with careful hand-crafted features and proper hyperparameter tuning can also deliver.

1. INTRODUCTION

The current trend of driving vehicles has now shifted to self-driving cars. While modern algorithms are not yet perfect, they are almost in the acceptable stage, especially with the fast advancement of state-of-the-art algorithms. One of the reasons of success here is the accurate recognition of these models on objects in front of them. Being able to determine the next course of action purely depends on what instruction it sees, and that is defined by traffic signs.

There has been a competition hosted and held at the IJCNN 2011 [1] where different models have been presented and compared with human recognition's accuracy of 99.81% in recognizing traffic signs. The best model used has been a Convolutional Neural Network that is supplemented by a handcrafted feature of HOG and has achieved 98.98% accuracy–a score that is as accurate as a human. It is already acknowledged that advanced and deep learning models can achieve this range of scores if trained with enough data, but can classical machine learning models achieve this level of accuracy?

2. OBJECTIVES

The objective of this study is to attempt to optimize models using only classical machine learning methods but with the aid of handcrafted features and determine the effects of these features. The effect of these results will be quantified by globally accepted metrics such as accuracy and macro F1 scores. The results between different models will also be compared and what model is better given the type of features it has.

3. METHODOLOGY

The entire end-to-end data pipeline can be summarized in Figure 1. We have a dataset composed of different traffic sign images where different transformations are performed to generate features. Principal Component Analysis is then applied to reduce the dimensionality of the generated features and then scaled using a Standard Scaler for more efficient training using different Machine Learning models. Each of these models is trained based on the hyperparameters dictated by the validation set. Once the models are trained, their accuracy and F1 scores are recorded and compared with each other to see how well each transformation performs on different models. Note here that the blocks with asterisks denote different configurations that can be changed. More of this will be discussed in the succeeding sections.

3.1 Dataset

The dataset used for this study is the German Traffic Sign Recognition Benchmark (GTSRB) which contains 39,209 images and has 43 different classes, each class represents a different kind of traffic sign. The number of images per class

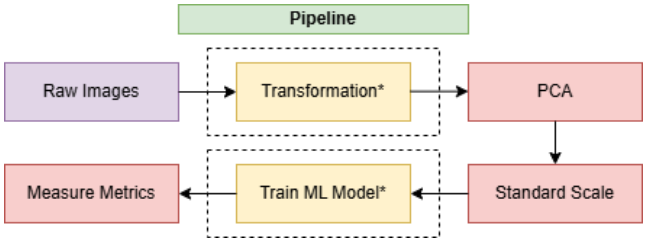

Figure 1: The main pipeline the data went to is shown here. Raw images are transformed to either of the four different transformations which are elaborated in Figure 4. To train efficiently, the features of the transformed data are then reduced using PCA and then are re-scaled for faster training. These are then trained on four kinds of models where they are then compared by accuracy and F1 score. Note that the phases with asterisks denote modularized phases, meaning they can be replaced depending on the configuration of interest.

seems somewhat imbalanced as shown in Figure 2. To imagine what each class holds, Figure 3 shows a sample of the different images per class. All images contain RGB data and hence have 3 dimensions. However, images vary in height and width where some images have a height range from 25 to 225 pixels and some have a width range from 25 to 243 pixels. Nonetheless, these raw images will be preprocessed to have consistent 64x64 pixels for length and width.

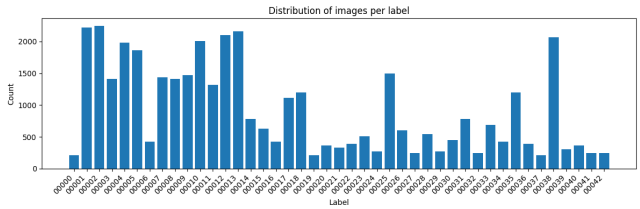

Figure 2: Shown here is the distribution of several images per class. The counts range from around 200 to 2200 where the average number of images lies around 900.

For this study, the dataset is divided into 70-30 with 70% for training and 30% for testing. The training dataset is further divided into 90-10 with 90% for the actual training while the 10% is used for validation. This validation partition is used for hyperparameter tuning to determine what is the best set of hyperparameters for a given model. For both splits, the dataset is stratified such that the distribution of the classes is maintained within the training, validation, and test splits.

3.2 Transformations

This part is the main focus of the paper. Four (4) different scenarios were considered, namely, Flatten Transformation, HOG Transformation, LBP Transformation, and Combined Transformation. See Figure 4 on the different steps that occur for each transformation.

3.2.1 Flatten Transformation

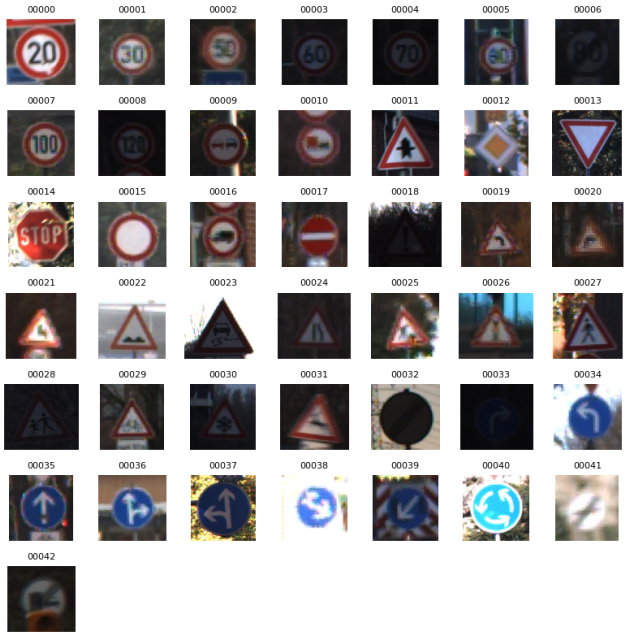

Figure 3: Each class contains a specific type of traffic sign which are either speed limits, prohibitory signs, warning signs, mandatory signs, priority signs, etc.

In this transformation, we simply retained the 64x64x3 dimensions of the image and then flattened it to obtain 12,288 features. Standard Scaling is then applied such that each feature will have mean zero and unit variance. Figure 5 displays a simplified version of how this is performed.

3.2.2 HOG Transformation

Histogram of Gradients, more popularly known as HOG, is a popular method in computer vision, especially in human detection problems. Our images are first transformed to grayscale–the average of the RGB vector. The gradients of each pixel are calculated from the differences of the adjacent pixels. Using these differences, the orientation is calculated and a histogram of these orientations is created. A full discussion of the method can be found in [2].

Before the histogram is created, Figure 6 shows the visualization of the gradients. The calculations are performed using the scikit-image package [3] using its *hog* method. For this study, only a certain configuration of HOG is used where the histogram uses 9 bins for orientation, pixels per cell is 8x8, cells per block is 8x8, and block normalization uses the 'L2-Hys'. In this case, the Standard Scale is also applied.

3.2.3 LBP Transformation

Local Binary Patterns are typically used in handwritten-recognition and face-recognition problems. Meaningful features are generated from the LBP as the local texture is measured by comparing a pixel with its neighboring pixel. That comparison is recorded as a binary number depending if that pixel is greater or less than the center pixel. These binary numbers are then stored into a binary string and then each transition from 0 to 1 or vice versa is counted. Full computation steps are detailed further in [4].

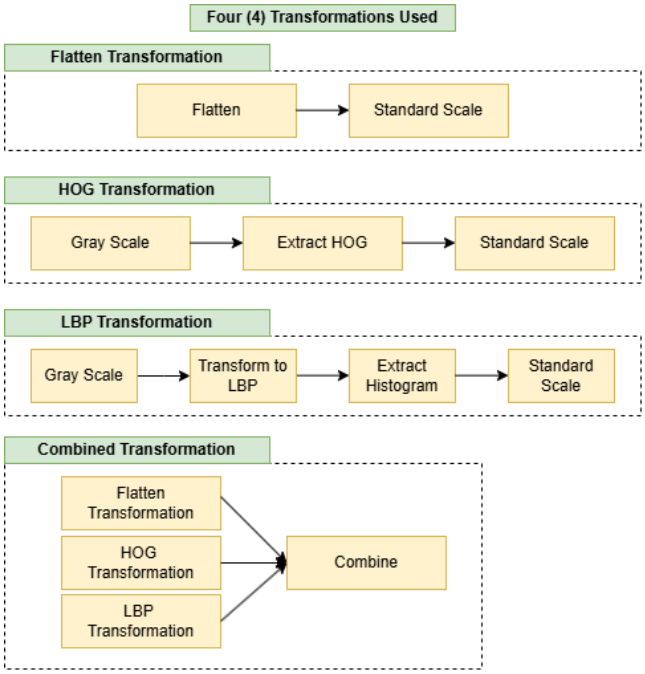

Figure 4: Here are the 4 transformations used, namely the Flatten Transformation, HOG Transformation, LBP Transformation, and Combined Transformation, each offering ways to craft features to be used for training the model.

Like HOG, images are first transformed to grayscale, and then transformation to LBP is performed using scikit-image with the *local_binary_pattern* method. Similarly, only a single configuration is performed, namely 8 of circularly symmetric neighbor set points, 1 pixel of circle radius, and a uniform method. However for this case, the scikit-image does not automatically provide a histogram, instead, the histogram is manually created by dividing the returned matrix into cells, with each cell generating its histogram. Lastly, the Standard Scale is still applied.

To visualize how the LBP helps in creating useful features, Figure 7 shows a comparison of an original image with an image transformed using a Local Binary Pattern.

3.2.4 Combined Transformation

The last scenario for the transformation is just a combined feature detailed from 3.2.1, 3.2.2, and 3.2.3. It is of interest whether combining useful features will generate more information for the machine learning models. While this will greatly increase the number of overall features, the next step of the pipeline will solve this issue.

3.3 Dimensionality Reduction

The different transformations discussed have quite a handful number of features, especially for the case described in Subsubsection 3.2.4. Given the length of the dataset, training this would take some time, hence Principal Component Analysis with 95% of the explained variance still intact is used. This significantly reduces the number of features to speed up training and reduce risks of overfitting while still

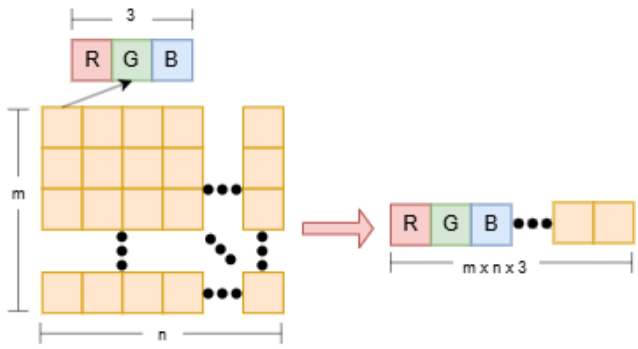

Figure 5: This transformation can be thought of as cutting each of the images horizontally and then concatenating the strips on each end. However, remember that each element of the image matrix is a vector of length 3, hence the final feature length is the product of pixels in length and width and 3.

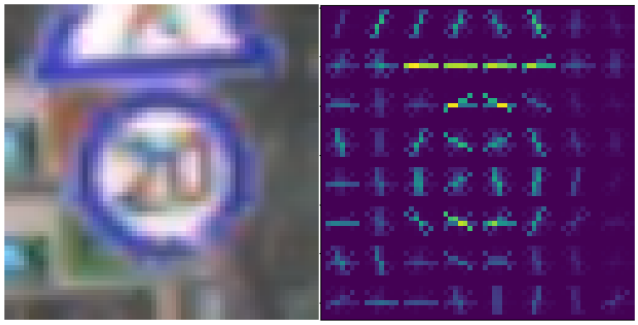

Figure 6: In the left is the original traffic sign image with its HOG visualization on the right. It can be observed that the HOG visualization can capture edges since it captures major differences in grayscale values.

maintaining enough information so the ML model can still pick up the necessary patterns.

Although prior to PCA, the model has already undergone Standard Scaler since PCA works best if data is properly scaled, the PCA-transformed data will undergo another instance of Standard Scaler. This is because the data produced by PCA rescales the data to a non-zero mean and non-unit variance. When used on some ML models, this may take some time hence it is decided to re-perform scaling. Additionally, note that PCA here is fitted only on the training data. The fitted PCA is then used on the validation and test dataset.

3.4 ML Models

Each of the different transformations will be trained on each of the machine learning models discussed in the subsequent sections. There are 4 models selected for this study, therefore a total of 16 transformation + model combinations were identified for this study. The selected models are namely, Stochastic Gradient Descent Classifier (SGD), Multilayer Perceptron (MLP), Support Vector Machine (SVM), and Extreme Gradient Boosting (XGBoost). The first 3 models

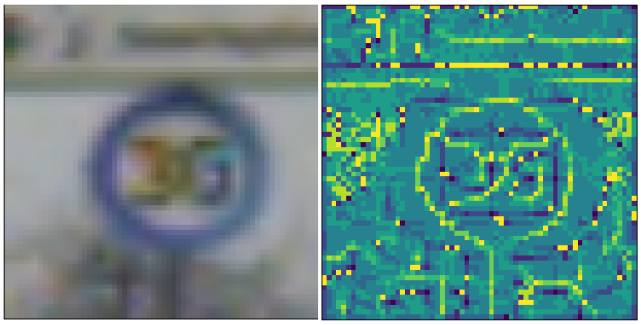

Figure 7: In the left is the original traffic sign image with its LBP visualization in the right. It can be observed that the LBP visualization can capture intensity transition around a given cell.

are implemented using Scikit-Learn [5] while XGBoost [6] is implemented via its own Python API package.

To ensure that the model is the most optimized, hyperparameter tuning is performed via randomized search. Each model will randomly search across a given grid of hyperparameters where the selected hyperparameters to be searched are believed to affect a given model the most i.e., *activation*, *alpha*, *hidden_layer_size*, etc. for an MLP model. The model with the randomly selected hyperparameters is then trained on the training data set and validated on the validation dataset. The combination of hyperparameters that best gives the macro F1 score will be the chosen model hyperparameters.

3.4.1 SGD

Stochastic Gradient Descent Classifier or SGD is a linear classifier that uses Stochastic Gradient Descent for training. The corresponding underlying model used depends on the selected loss function hyperparameter with *hinge* for linear SVM, *log_loss* for logistic regression, *modified_huber* for a robust model that can handle outliers well, etc. This model was selected because of its linear nature hence it is fast to train and serves as a good baseline for comparison.

3.4.2 MLP

For this study, the Multilayer Perceptron used here will be limited to only 2 hidden layers. This model performs a weighted sum of the features and is then fed to a nonlinear activation function in the next layer. This next layer then repeats the same steps until it reaches the outer layer where it performs a softmax prediction. Any error is continuously corrected by the backpropagation algorithm.

3.4.3 SVM

The Support Vector Machine in this study uses the Radial Basis function kernel to implicitly map the data to another space where there exists a hyperplane that can separate the data.

3.4.4 XGBoost

Extreme Gradient Boosting is an improved Gradient Boosting Machine model where it uses multiple trees to correct the initial predictions depending on the computed pseudo-residuals. This model uses different techniques to be more robust to outliers and be able to generalize well and still scale well with large data sizes.

3.5 Metrics

The different models are hyperparameter-tuned to give the best macro F1 score. F1 score is the harmonic mean of the precision and recall where precision is a measure of a model's capability to capture positives with minimal false positives while recall is the capability to capture positives with minimal false negatives. F1 score is the metric that is able to quantify both stories. In the case of multiclass problems, there is an F1 score for each class, hence we defined the macro F1 score as the average F1 score, independent of the distribution of each class, as it is assumed that each sign is equal with other signs.

Another metric that is looked at is the accuracy. The accuracy is simply the correct classification over the total number of items to classify.

4. EXPERIMENTAL RESULTS

The experiment is performed 16 times, with each performed on a given Transformation and a given ML model. The resulting accuracy for each transformation + model pair is recorded and plotted in Table 1 and Figure 8. The same is also performed for their macro F1 score as seen in Table 2 and Figure 9.

Accuracy	SGD	MLP	SVM	XGBoost
Flatten	88.71%	96.65%	95.38%	92.29%
HOG	96.01%	96.99%	97.86%	94.18%
LBP	83.83%	84.76%	89.12%	81.75%
Combined	98.40%	98.89%	99.09%	96.40%

Table 1: Shown are the test accuracy scores for each model against the different kinds of transformations performed on the data. Highlighted are the highest accuracy scores for each transformation. As observed, using HOG as a feature already gives strong performance but adding the original data with LBP further improves the results! While other models display great accuracy with the combined transformations, the SVM is truly the most impressive out of the four compared models.

Macro F1 Score	SGD	MLP	SVM	XGBoost
Flatten	88.06%	96.32%	95.07%	91.71%
HOG	96.96%	97.59%	98.51%	94.11%
LBP	83.88%	84.80%	90.49%	79.20%
Combined	98.60%	99.02%	99.04%	95.95%

Table 2: Shown are the test macro F1 scores for each model against the different kinds of transformations performed on the data. It is shown that the F1 scores closely follow the same trend as that of the accuracy scores, with the SVM and the combined transformation showing the best overall results.

The confusion matrix of the test result using the best model is presented in Figure 10.

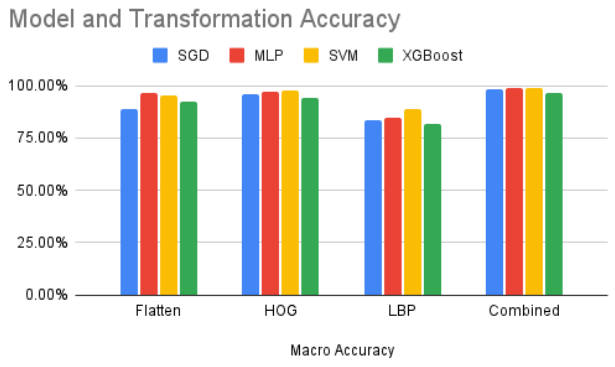

Figure 8: Bar chart here is the visualization of Table 1 where it shows the different accuracy scores per model and transformation.

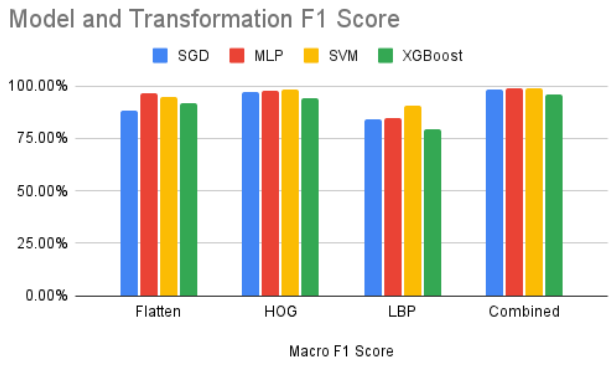

Figure 9: Bar chart here is the visualization of Table 2 where it shows the different accuracy scores per model and transformation.

5. ANALYSIS AND DISCUSSION OF RESULTS

Based on the experimental results, using Flatten Transformation is already able to give us good results of 96.65% accuracy and 96.32% macro F1 score–and this is without any further additional feature crafting, only flattening the data and reducing its dimensions using PCA. In this scenario, the Multilayer Perceptron is the best model that can classify the kind of traffic sign, with SVM the close second. XGBoost gives relatively mediocre results and SGD has the lowest score for both metrics.

Next, when using purely Histogram of Gradients, it has greatly improved the results regardless of the used model. The jump for SGD is astounding and significantly improved both XGBoost and SVM while the least improvement is for MLP, but still an improvement! For this case, SVM has overtaken MLP as being the better model.

For the case of purely utilizing the Local Binary Pattern, we can see that these showed the least preferred results out of the four kinds of transformations. The results here, while not bad, are relatively distant from the two first scenarios. However, it can be seen from this result that SVM is able to generalize better with hand-crafted features.

Lastly, an attempt to combine the three scenarios was tested, and it is shown that this is the best Transformation so far. It can utilize the original features together with 2 sets of handcrafted features. While dimensionally reduced, it is seen that it contains the most useful information for classification hence the improved results for all models. This time, MLP and SVM are now almost comparable in results with SVM the winner by a small margin.

It can be interpreted that without any handcrafted features, an MLP, even with low hidden layers, can capture patterns within images. But with the proper handcrafted features, SVM can outperform MLP results. By combining all the relevant features, SVM is the better model out of the four!

6. CONCLUSION

An interim finding of this study is that when using Multilayer Perceptron on the original data, it is able to generalize well. However, once additional informative features are used and added, Support Vector Machine models now start to take the lead. This can be interpreted such that supplementing our original data with meaningful features helps SVM's RBF kernel map the original dataset to a more separable space.

According to the [1], the best model using a Convolutional Neural Network achieved an accuracy of 98.98%. While there could have been more studies using more advanced models that are trained on the same dataset and have met better results, it cannot be denied that classical machine learning models can also achieve a good performance provided it is properly engineered with meaningful and informative features. This is backed up by the results shown by the Support Vector Machine that uses original data with HOG and LBP, and then dimensionally reduced by PCA.

Further improvements to the study can be performed such as using other ML algorithms not defined here. There are also other feature engineering techniques that can be used that can be incorporated. Additionally, HOG and LBP tackled here only used a single configuration. Another improvement is to consider the parameters of HOG and LBP as hyperparameters that can be optimized further. Lastly, the ratio of explained variance for PCA can also be searched, or another dimensionality reduction technique can be used.

While state-of-the-art models are popular for image recognition problems, there are still cases where classical machine learning models are usable.

7. REFERENCES

[1] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "The german traffic sign recognition benchmark: A multi-class classification competition," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2011, pp. 1453–1460. [Online]. Available: https://ieeexplore.ieee.org/document/6033395

[2] N. Dalal and B. Triggs, "Histograms of oriented

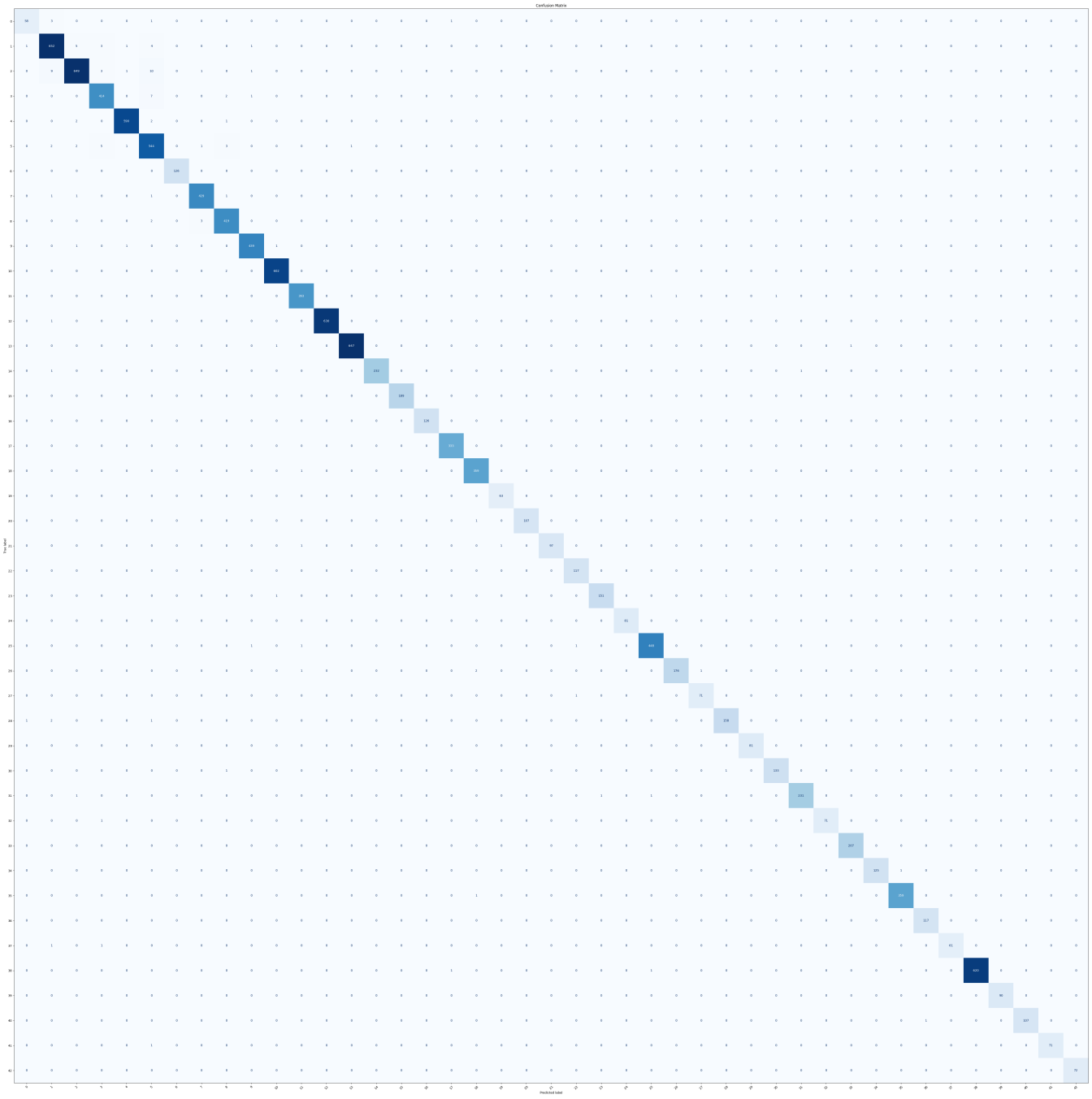

Figure 10: This is the confusion matrix for the SVM model trained on the data that uses that Combined Transformation and tested on the test data. It can be observed that it is able to classify well with minimal misclassifications. The majority of the remaining misclassifications are those within the speed limits themselves, showing that while the model can identify the general shape, there is room for improvement for similarly shaped signs with almost similar symbols.

gradients for human detection," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, IEEE. San Diego, CA, USA: IEEE, 2005, pp. 886–893.

[3] S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, T. Yu, and the scikit-image contributors, "scikit-image: image processing in python," *PeerJ*, vol. 2, p. e453, 2014.

[4] T. Ojala, M. Pietikäinen, and D. Harwood, "Texture classification using local binary patterns," in *Proceedings of the European Conference on Computer*

Vision (ECCV). Springer, 1996, pp. 607–610.

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.

[7] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "German traffic sign recognition benchmark (gtsrb) dataset," 2011, accessed: 2025-05-25. URL: https://sid.erda.dk/public/archives/daaeac0d7ce1152aea9b61d9f1e19370/published-archive.html.